Mini Project Report

ON

# "Evaluate performance enhancement of parallel Quicksort Algorithm using MPI"

Submitted to
SAVITRIBAI PHULE PUNE UNIVERSITY

In Partial Fulfillment of the Requirement for the Award of

B.E COMPUTER ENGINEERING
SEMESTER VIII

SUBMITTED BY


| | |
|---|---|
| Anjali Santosh Barge | Exam No.: |
| Neha Nana Dhurgude | Exam No.: |
| Dhanashree Tanaji Jadhav | Exam No.: |

Under the Guidance of

Prof. Jayashri Mankar

DEPARTMENT OF COMPUTER ENGINEERING

ALARD COLLEGE OF ENGINEERING AND MANAGEMENT

MARUNJI, PUNE-57

(AFFILIATED TO SAVITRIBAI PHULE PUNE UNIVERSITY)


2025 – 2026

# CERTIFICATE

This is to certify that the Mini Project report entitles

"Evaluate performance enhancement of parallel Quicksort Algorithm using MPI"

Submitted by

| | |
|---|---|
| Anjali Santosh Barge | Exam No.: |
| Neha Nana Dhurgude | Exam No.: |
| Dhanashree Tanaji Jadhav | Exam No.: |

Are bonafide students of this institute, and the work has been carried out by them under the supervision of Prof. Jayashri Mankar and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the B.E. Computer Engineering.

**Prof. Jayashri Mankar**
Project Guide
Dept. of Computer Engg.

**Prof. Dipali Mane**
Head of Department
Dept. of Computer Engg.

Place:
Date:

## Acknowledgements

It is my privilege to express my sincerest regards to my Project guide, Prof. Jayashri Mankar, for her valuable inputs, able guidance, encouragement, whole-hearted cooperation and constructive criticism throughout the duration of my project. I deeply express my sincere thanks to my Head of Department Prof. Dipali Mane for encouraging and allowing us to present the project on the topic "Evaluate performance enhancement of parallel Quicksort Algorithm using MPI." at my department premises. I take this opportunity to thank all my faculties who directly or indirectly helped my project. I pay my respect and love to my parents and all other family members and friends for their love and encouragement throughout my career. Last but not the least I express my thanks to my friends for their cooperation and support.

# Abstract

High Performance Computing (HPC) enables efficient processing of large computational tasks through parallel execution. Sorting large datasets is a common operation, and improving its performance using parallel algorithms is an important application of HPC.

This project focuses on evaluating the performance of the Quicksort algorithm using the Message Passing Interface (MPI). The implementation distributes a randomly generated dataset among multiple processes using MPI_Scatter. Each process performs local sorting using the standard quicksort method, and the results are combined using MPI_Gather. The total execution time is measured using MPI_Wtime().

The program was executed using 1, 2, and 4 processes. The observed execution times were 0.2156 seconds, 0.1344 seconds, and 0.0974 seconds respectively. The corresponding speedups were 1.00, 1.60, and 2.21, indicating improved performance with parallel execution.

The results show that parallelization reduces execution time; however, the speedup is not linear due to communication overhead and process synchronization. This project demonstrates the practical implementation of MPI-based parallel computing and highlights its effectiveness in improving algorithm performance for larger datasets.

# Table of Contents


| Sr. No. | Topic | Page No. |
|---|---|---|
| 1 | Introduction | 1 |
| 2 | Project Overview | 2 |
| 3 | Methodology and Implementation | 3-4 |
| 4 | Code Description | 5-6 |
| 5 | Result and Analysis | 7 |
| 6 | Conclusion and References | 8 |

# 1. Introduction

Sorting is a fundamental operation in computer science with applications in database systems, data analytics, scientific simulations, and information retrieval. Efficient sorting is essential for improving the performance of many computational tasks, especially when dealing with large datasets. As the volume of data continues to grow in modern applications, traditional sequential sorting algorithms may become time-consuming and inefficient, creating a need for faster and more scalable solutions.

High Performance Computing (HPC) provides an effective approach to address these challenges by utilizing parallel processing techniques. Instead of executing tasks sequentially on a single processor, HPC enables multiple processes to execute simultaneously, thereby reducing computation time and improving overall system performance. Parallel computing is particularly useful for problems that can be divided into smaller independent tasks, such as sorting, matrix operations, and large-scale simulations. One of the most widely used standards for implementing parallel programs in distributed memory systems is the Message Passing Interface (MPI). MPI allows multiple processes to communicate with each other by sending and receiving messages, making it suitable for building scalable parallel applications.

Quicksort is a highly efficient sorting algorithm based on the divide-and-conquer paradigm. It works by selecting a pivot element, partitioning the dataset into smaller subsets, and recursively sorting these subsets. Due to its average-case time complexity of O(n log n), Quicksort is commonly used in practical applications. However, in its standard form, Quicksort operates sequentially, which limits its performance when dealing with very large datasets. By applying parallel computing techniques, the sorting process can be distributed across multiple processes, allowing different portions of the dataset to be sorted simultaneously, leading to improved performance.

In this project, the Quicksort algorithm is implemented using MPI to evaluate its performance in a parallel computing environment. The dataset is initially generated and distributed among multiple processes using MPI communication functions such as MPI_Scatter. Each process performs local sorting independently, and the sorted results are collected using MPI_Gather. The total execution time is measured using MPI_Wtime() for different numbers of processes, such as 1, 2, and 4. The performance of the parallel implementation is analyzed by comparing execution times and calculating speedup.

This work provides practical exposure to HPC concepts and demonstrates how parallel programming techniques can improve the performance of computational algorithms. It also emphasizes the importance of performance evaluation in understanding the behavior of parallel systems in real-world scenarios.

# 2. Project Overview

This project implements and evaluates a parallel version of the Quicksort algorithm using the Message Passing Interface (MPI). The primary goal is to study how distributing the sorting workload across multiple processes affects execution time and overall performance. Unlike a purely sequential approach, this implementation leverages multiple processes to perform computation simultaneously.

The system begins with the root process generating a large array of random integers. This dataset is then divided into equal-sized chunks based on the number of processes available. Using the `MPI_Scatter` operation, each process receives a portion of the data. This distribution ensures that the workload is shared among processes, enabling parallel execution.

Each process independently sorts its assigned subset using the standard quicksort logic (implemented via the C library function `qsort()`). Since the sorting operations occur simultaneously across processes, the computation time is reduced compared to sequential execution. After local sorting, the sorted segments are transmitted back to the root process using `MPI_Gather`.

Once all partial results are collected, the root process performs a final sorting step to merge the subarrays into a completely sorted dataset. The total execution time is measured using `MPI_Wtime()`, which provides high-resolution timing suitable for performance evaluation in parallel systems.

The implementation is tested with different numbers of processes (1, 2, and 4) to observe performance variation. By comparing execution times across these configurations, the speedup achieved through parallelization is calculated. This allows analysis of how efficiently the workload is distributed and highlights practical limitations such as communication overhead and synchronization delays.

This project demonstrates distributed parallel computing using MPI and emphasizes performance analysis to evaluate the effectiveness of parallel algorithms.

# 3. Methodology and Implementation

This section describes the methodology followed to implement the parallel Quicksort algorithm using MPI. The implementation is carried out step by step, starting from data generation to performance evaluation.

## 3.1. Data Generation

The dataset used in this project is a randomly generated array of integers. The root process is responsible for generating the input data. The size of the dataset is selected large enough to observe the impact of parallel execution on performance.

## 3.2. Initialization of MPI Environment

The MPI environment is initialized using MPI_Init(). Each process is assigned:

- A unique process ID using MPI_Comm_rank()
- Total number of processes using MPI_Comm_size()

This information is used to divide the workload among processes.

## 3.3. Data Distribution

The generated dataset is divided equally among all processes. The root process distributes data using the MPI_Scatter() function.

- Each process receives a portion of the array
- Workload is distributed evenly
- Enables parallel computation

## 3.4. Local Sorting

Each process independently sorts its assigned subset of data using the standard C library function qsort().

- Sorting is performed in parallel
- No dependency between processes during this phase
- Reduces computation time compared to sequential execution

## 3.5. Data Collection

After local sorting is completed, all sorted subarrays are sent back to the root process using MPI_Gather().

- Root process collects all partial results
- Ensures all data is available for final processing

## 3.6. Final Merging and Sorting

The root process performs a final sorting step on the combined data to ensure the complete dataset is properly ordered.

- Necessary because individual sorted chunks are not globally sorted
- Produces the final sorted output

## 3.7. Performance Measurement

The execution time is measured using MPI_Wtime()

- Start time recorded before computation

- End time recorded after completion
- Total execution time calculated

This process is repeated for different numbers of processes (1, 2, and 4) to evaluate performance.

## 3.8. Speedup Calculation

Speedup is calculated using the formula:

Speedup = Time (1 process) / Time (n processes)

This metric is used to evaluate the efficiency of parallel execution.

## 3.9. System Workflow

The overall workflow of the system is as follows:

1. Initialize MPI environment
2. Generate input data (root process)
3. Distribute data using MPI_Scatter
4. Perform local sorting in each process
5. Collect sorted data using MPI_Gather
6. Perform final sorting at root process
7. Measure execution time
8. Calculate speedup and analyze performance

# 4. Code Description

This section explains the implementation of the parallel Quicksort algorithm using C and MPI. The program is designed to distribute the sorting task across multiple processes and evaluate performance using parallel execution.

The program begins by importing the required libraries. The MPI library (mpi.h) is used for parallel communication, while standard libraries such as `stdio.h`, `stdlib.h`, and `time.h` are used for input/output operations, memory management, and random number generation.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

A comparison function is defined to support sorting using the qsort() function. This function compares two integer values and returns their difference.

```
int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}
```

The main function initializes the MPI environment. Each process identifies its unique rank (process ID) and the total number of processes using MPI communication functions.

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

The root process (rank 0) generates a large array of random integers. This dataset is used as input for the sorting operation.

```
if (rank == 0) {
    arr = (int*)malloc(n * sizeof(int));
    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }
}
```

The total dataset is divided among all processes. Each process allocates memory to store its local portion of the data.

```
int local_n = n / size;
int* local_arr = (int*)malloc(local_n * sizeof(int));
```

Execution time measurement is started using MPI_Wtime() before the parallel operations begin.

```
double start = MPI_Wtime();
```

The dataset is distributed among processes using the MPI_Scatter() function. Each process receives an equal share of the data.

```
MPI_Scatter(arr, local_n, MPI_INT,
        local_arr, local_n, MPI_INT,
        0, MPI_COMM_WORLD);
```

Each process performs local sorting using the qsort() function. This step executes in parallel across all processes.

```
qsort(local_arr, local_n, sizeof(int), compare);
```

After local sorting is completed, the sorted subarrays are collected back to the root process using MPI_Gather().

```
MPI_Gather(local_arr, local_n, MPI_INT,
        arr, local_n, MPI_INT,
        0, MPI_COMM_WORLD);
```

The root process performs a final sorting step to ensure the entire dataset is correctly ordered. The execution time is calculated and displayed.

```
if (rank == 0) {
   qsort(arr, n, sizeof(int), compare);

   double end = MPI_Wtime();
   printf("Time taken with %d processes: %f seconds\n", size, end - start);
}
```

Finally, allocated memory is freed, and the MPI environment is terminated using MPI_Finalize().

```
free(local_arr);
if (rank == 0) free(arr);


MPI_Finalize();
```

This implementation illustrates how MPI enables parallel execution by distributing computation across multiple processes, thereby improving performance for large datasets.

# 5. Result and Analysis

The performance of the parallel Quicksort algorithm was evaluated using MPI by measuring execution time for different numbers of processes. The program was executed in a WSL (Ubuntu) environment using OpenMPI on a dataset of 1,000,000 randomly generated integers. Execution time was measured using MPI_Wtime().

The observed results are shown below:

| Number of Processes | Execution Time (seconds) | Speedup |
|---|---|---|
| 1 | 0.2156 | 1.00 |
| 2 | 0.1344 | 1.60 |
| 4 | 0.0974 | 2.21 |

*Note: Execution time may vary slightly between runs due to system conditions.*

The graphical representation of execution time versus number of processes is shown below.

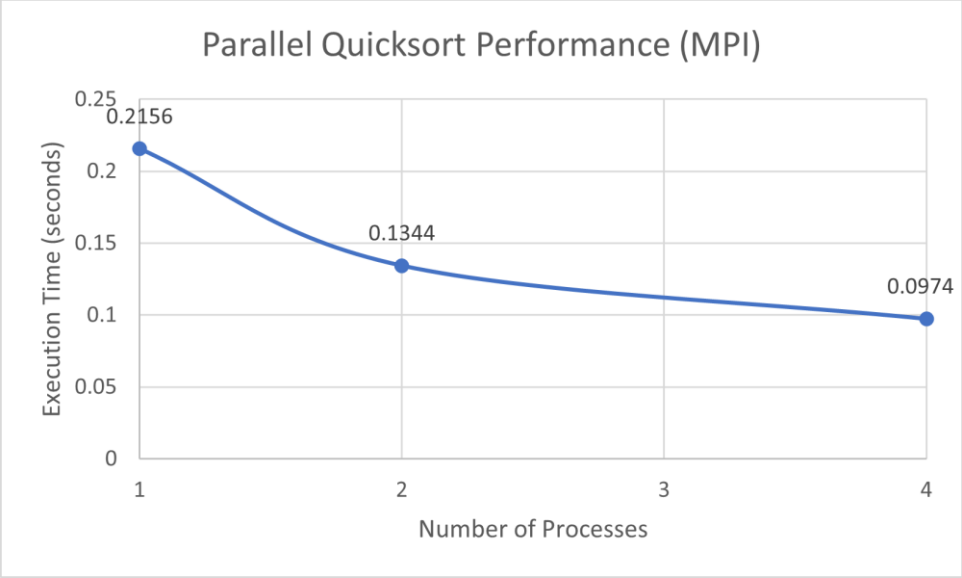


Figure 1: Execution Time vs Number of Processes

The results clearly show that execution time decreases as the number of processes increases, indicating improved performance through parallelization. A significant reduction in execution time is observed when increasing from 1 to 2 processes, while the improvement from 2 to 4 processes is comparatively smaller.

The speedup achieved is not linear. This is due to factors such as communication overhead during data distribution (MPI_Scatter) and collection (MPI_Gather), synchronization delays between processes, and limitations of the system hardware. As the number of processes increases, the communication cost begins to impact the overall performance.

Overall, the results demonstrate that parallel Quicksort using MPI improves execution efficiency, although performance gains are limited by practical constraints in parallel systems.

# 6. Conclusion and References

This project demonstrates the implementation of the Quicksort algorithm using MPI to achieve parallel execution. By distributing the dataset across multiple processes and performing local sorting, the overall execution time is reduced compared to sequential execution.

The experimental results show that increasing the number of processes improves performance, as observed from the reduction in execution time and corresponding speedup. However, the speedup is not linear due to factors such as communication overhead, synchronization delays, and system limitations. These results highlight the practical challenges involved in parallel computing.

This project provides practical exposure to MPI-based programming and demonstrates how parallelization improves algorithm performance. Further improvements can be achieved by using larger datasets, optimizing load distribution, and exploring advanced parallel sorting techniques.

**References**


1. Open MPI Project. (2024). *Open MPI Documentation*. Available at: https://www.open-mpi.org
2. Message Passing Interface Forum. (2023). *MPI: A Message-Passing Interface Standard Version 4.1*.
3. Gropp, W., Lusk, E., & Skjellum, A. (2022). *Using MPI: Portable Parallel Programming with the Message Passing Interface*. MIT Press.
4. Rauber, T., & Rünger, G. (2022). *Parallel Programming: For Multicore and Cluster Systems*. Springer.
5. Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2021). *Introduction to Parallel Computing* (2nd ed.). Pearson.
6. Pacheco, P. (2021). *An Introduction to Parallel Programming*. Morgan Kaufmann.
7. Chapman, B., Jost, G., & van der Pas, R. (2021). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.
8. Hennessy, J. L., & Patterson, D. A. (2021). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.
9. Sanders, P., & Kandrot, E. (2021). *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley.
10. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
11. Quinn, M. J. (2020). *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill.
12. Wilkinson, B., & Allen, M. (2020). *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers*. Pearson.
13. Dongarra, J., et al. (2020). *High Performance Computing: Modern Systems and Practices*. Morgan Kaufmann.
14. OpenMP Architecture Review Board. (2020). *OpenMP Application Programming Interface Version 5.0*.
15. NVIDIA Corporation. (2020). *CUDA C Programming Guide*. Available at: https://docs.nvidia.com/cuda